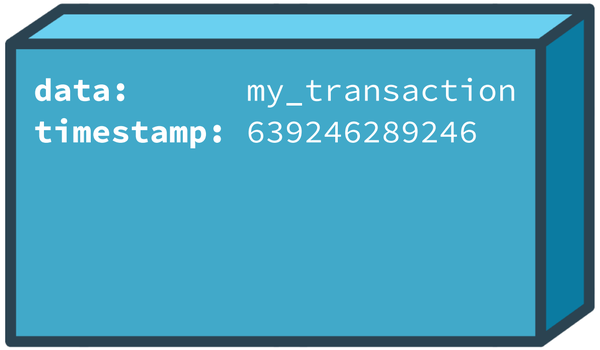
Lecture 2

Lecture table of contents

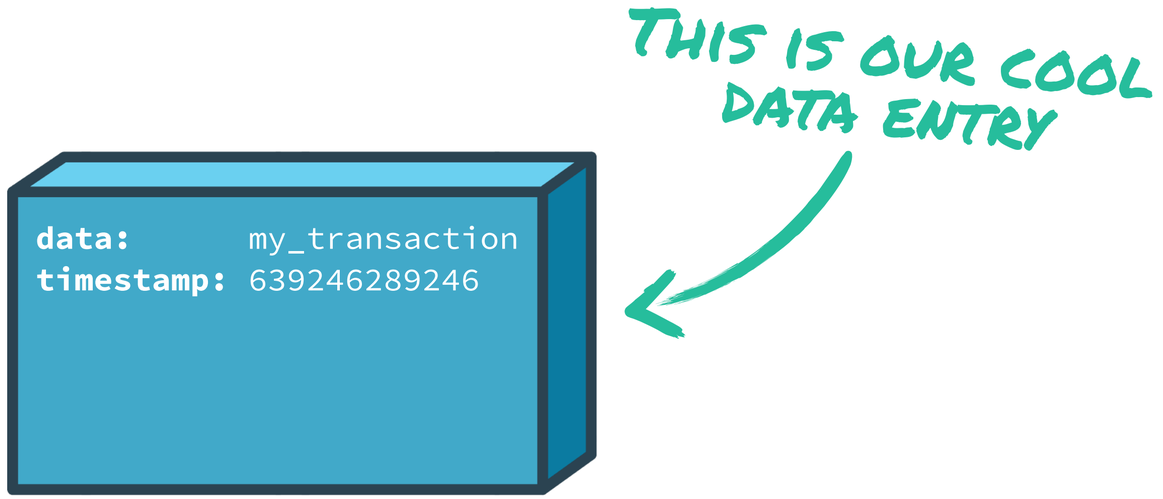
- Learn about Blockchain
- Learn about hashing
- Learn about or coding standards

Blockchain Basics

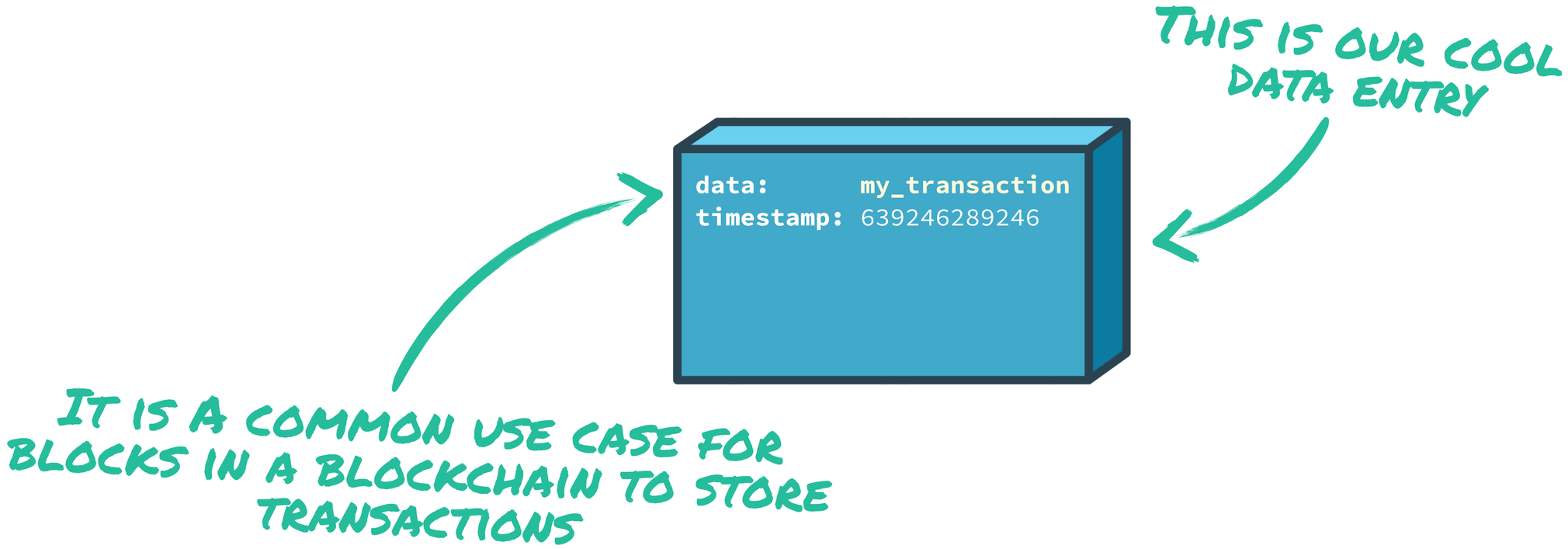
Let's start with a block



—Let’s start with a block



Let's start with a block



OpenElements

Hedera | SWIRLDS LABS

Let's start with a block

WAIT

This is our cool data entry

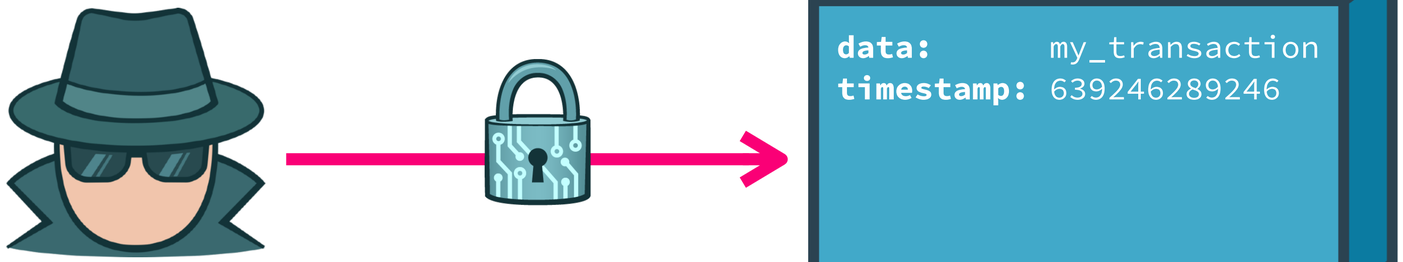
It is a common use case for blocks in a blockchain to store transactions

WAIT

I can store the exact same information by simply using

- a SQL DB
- Java Objects
- Java Records
- JSON
- a simple text file
- JPA
- Hibernate
- MongoDB
- the Cloud
- Redis

Let's start with a block



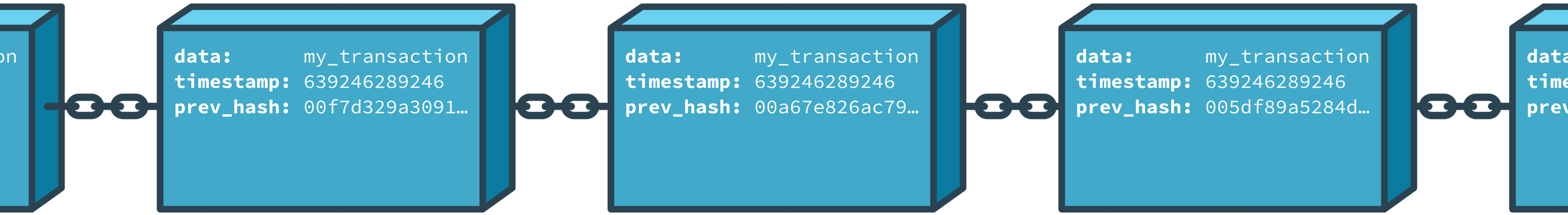
I want to **mutate** the transaction and add 1,000,000 € to my account

We want to secure the data and make it impossible to change any transaction information

OpenElements

Hedera | SWIRLDS LABS

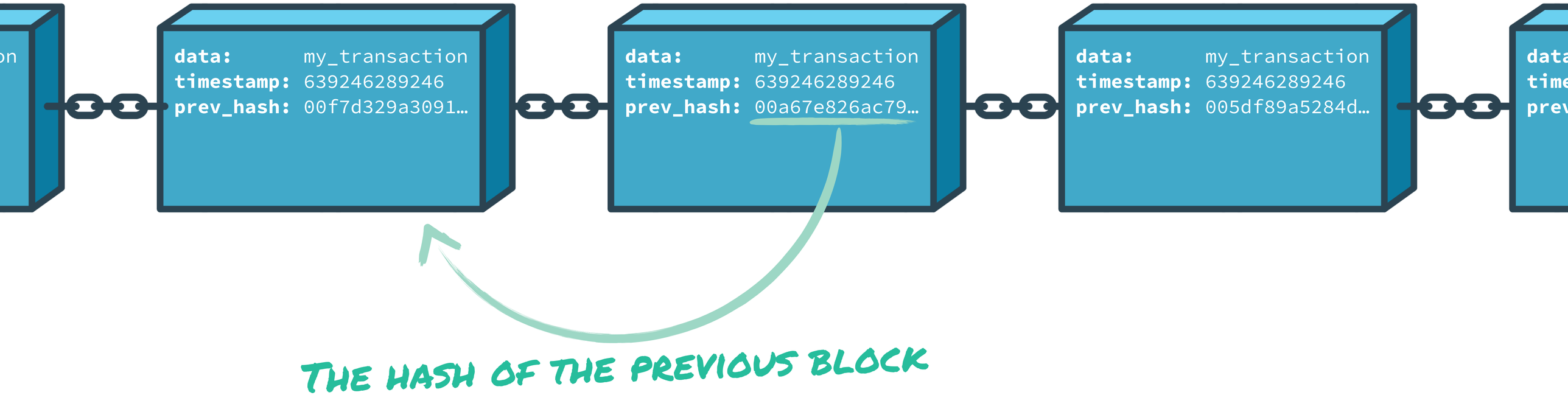
—… the chain comes into play



OpenElements

Hedera | SWIRLDS LABS

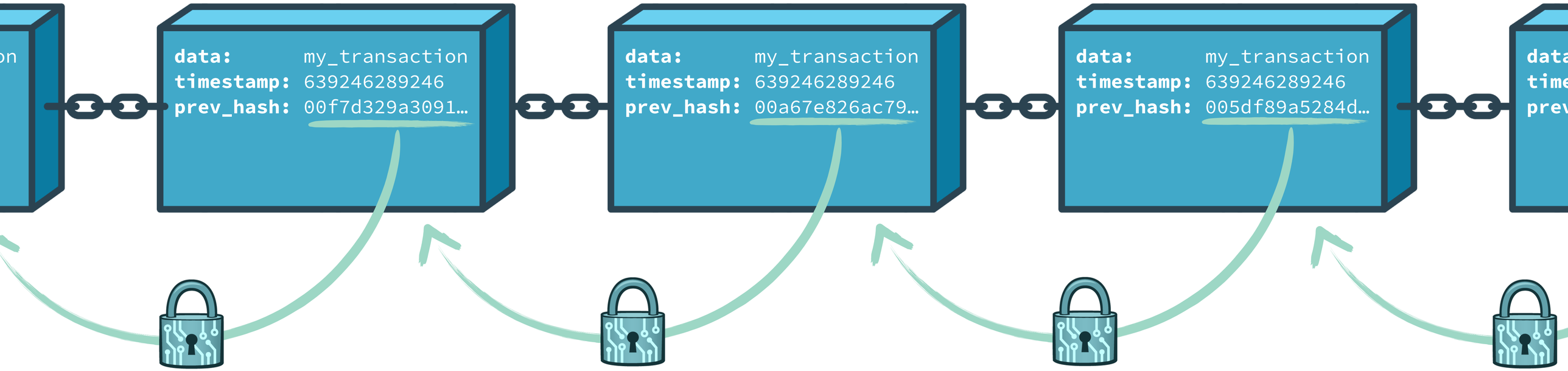
—The Blockchain



OpenElements

Hedera | SWIRLDS LABS

—The Blockchain

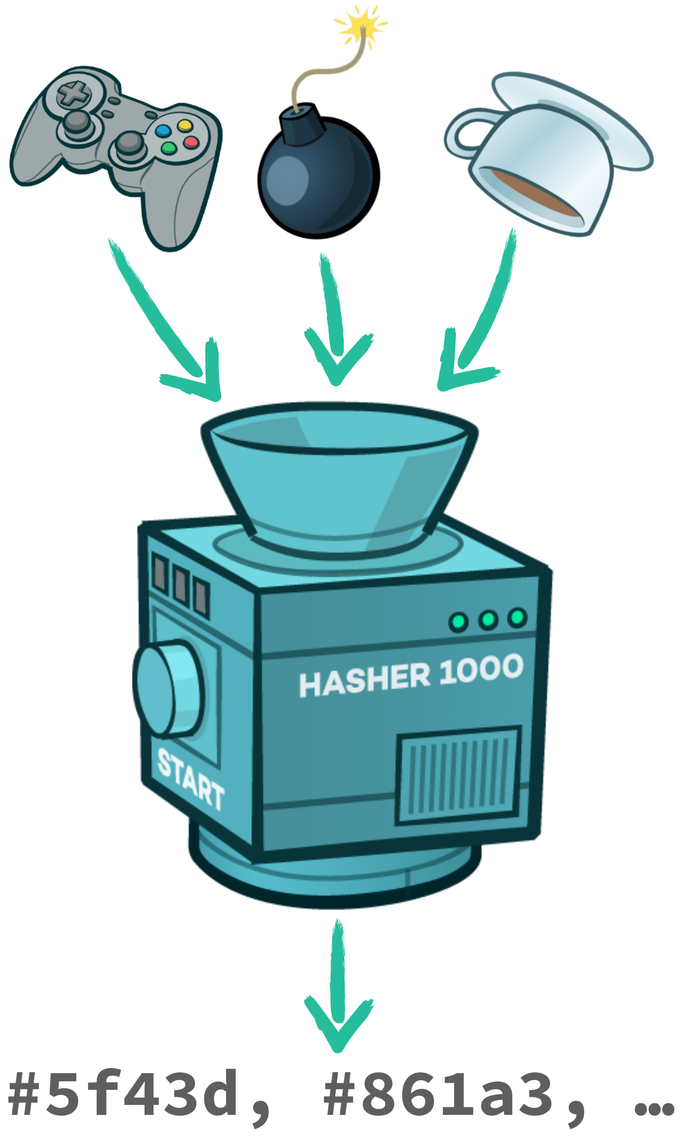


EACH BLOCKS DEFINES THE VALIDITY OF ALL PREVIOUS BLOCKS

Hash Functions

Hash Functions

- Map data to a hash
- Input is normally (large) data of any size and type
- Output is normally fixed-size data



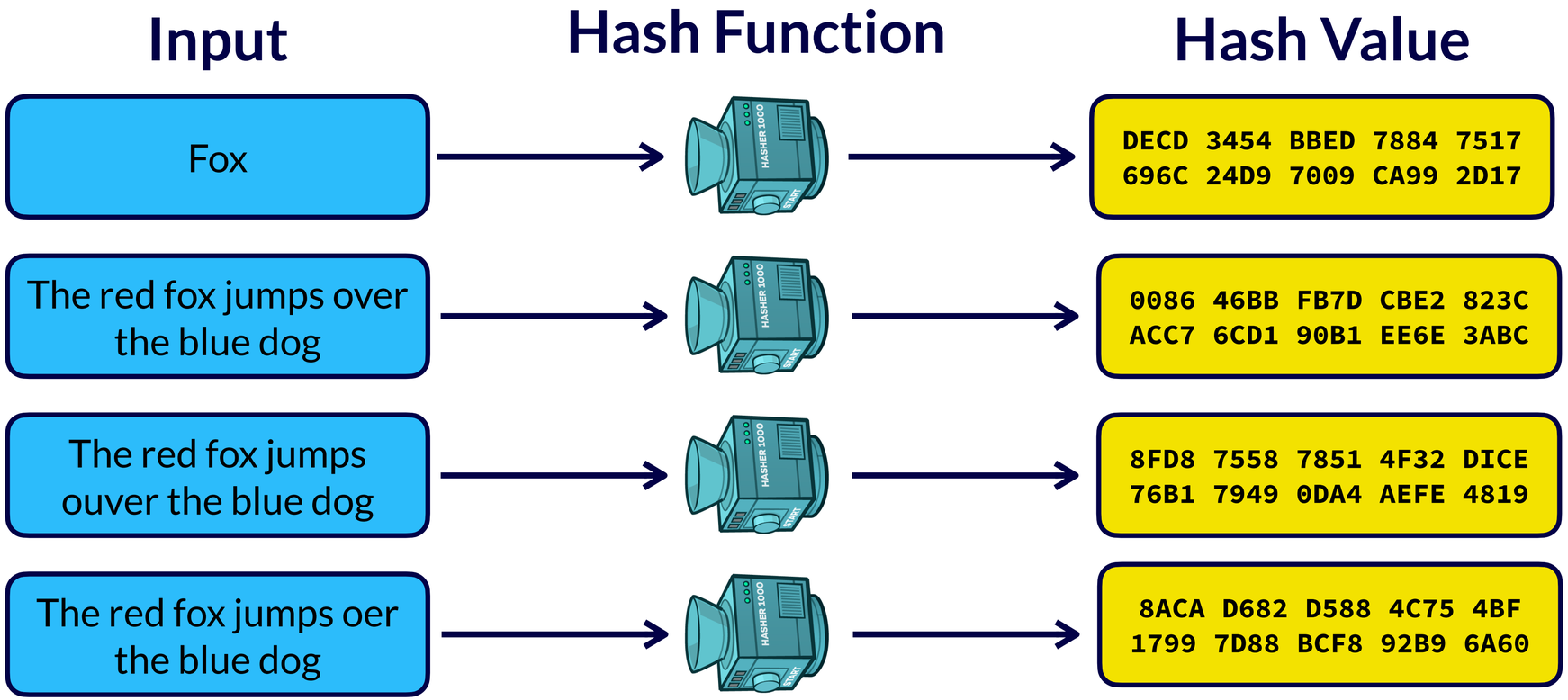
Specification of Hash Functions

- **Uniformity:** map the inputs as evenly as possible over its output range
- **Efficiency:** Calculation must be fast without using much memory
- **Deterministic:** for a given input value it must always generate the same hash value.
- **Surjection:** Every value of the hash range should be a reachable result
- **One-way-function:** Not possible to reverse the computation

Secure Hash Functions

- **Data Integrity:** Ensures data integrity by generating a unique hash output for a given input, making any alteration in data easily detectable.
- **One-Way Function:** Functions that are easy to compute in one direction but computationally infeasible to reverse.
- **Collision Resistance:** Designed to minimize the probability of two different inputs producing the same hash output.
- **Fixed Output Length:** Produces a fixed-length output, regardless of the length of the input, aiding in data uniformity and consistency.

Example of Hashing



—Usecases of Hashing

- **Checksum:** check a large download by comparing the checksum over the bytes of the file
- **Passwort storage:** Store the hash of a password instead of storing the real password
- **Hashtable:** Store large data collections in hash based tables / buckets

Digit Sum

- The most simple hash function

—Digit Sum

- The most simple hash function

Definition [edit]

Let n be a natural number. We define the **digit sum** for base $b > 1$, $F_b : \mathbb{N} \to \mathbb{N}$ to be the following:

$$F_b(n) = \sum_{i=0}^{k} d_i$$

where $k = \lfloor \log_b n \rfloor$ is one less than the number of digits in the number in base b, and

$$d_i = \frac{n \bmod b^{i+1} - n \bmod b^i}{b^i}$$

is the value of each digit of the number.

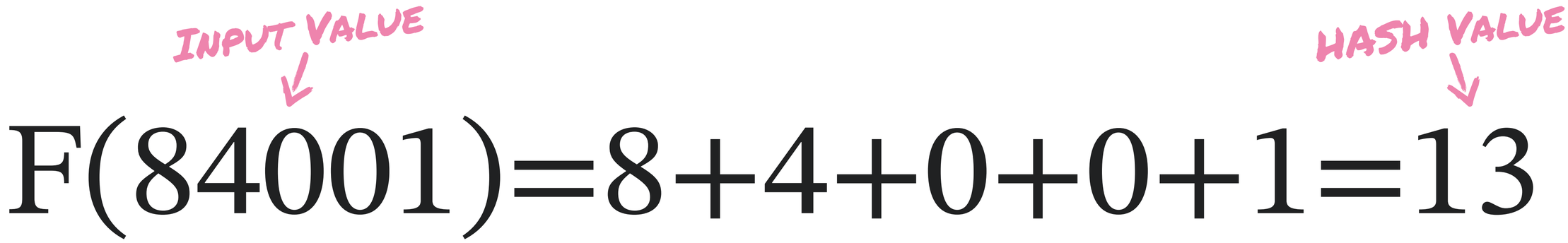
—Digit Sum

- The most simple hash function

$$F(84001)=8+4+0+0+1=13$$

—Digit Sum

- The most simple hash function



—Digit Sum

- The most simple hash function

$$F(84001)=8+4+0+0+1=13$$



ONLY COMPUTABLE IN ONE DIRECTION

—Digit Sum

- The most simple hash function

$$F(84001) = 8 + 4 + 0 + 0 + 1 = 13$$

Calculation is really fast!

—Digit Sum

- The most simple hash function

$$F(84001)=8+4+0+0+1=13$$

UNIFORMITY + SURJECTION ARE NOT REALLY GOOD...

Hands on lesson

- Create a small program that calculates a digit sum
- Digit sum must be printed as string
- Digit sum must always be 8 digits long

MD5 Hash Function

- Message Digest Algorithm 5
- Widely used cryptographic hash function that produces a 128-bit (16-byte) hash value
- Previously used in SSL certificates, password hashing, and other cryptographic applications

MD5 Hash Function

- Message Digest Algorithm 5
- Widely used cryptographic hash function that produces a 128-bit (16-byte) hash value
- Previously used in SSL certificates, password hashing, and other cryptographic applications

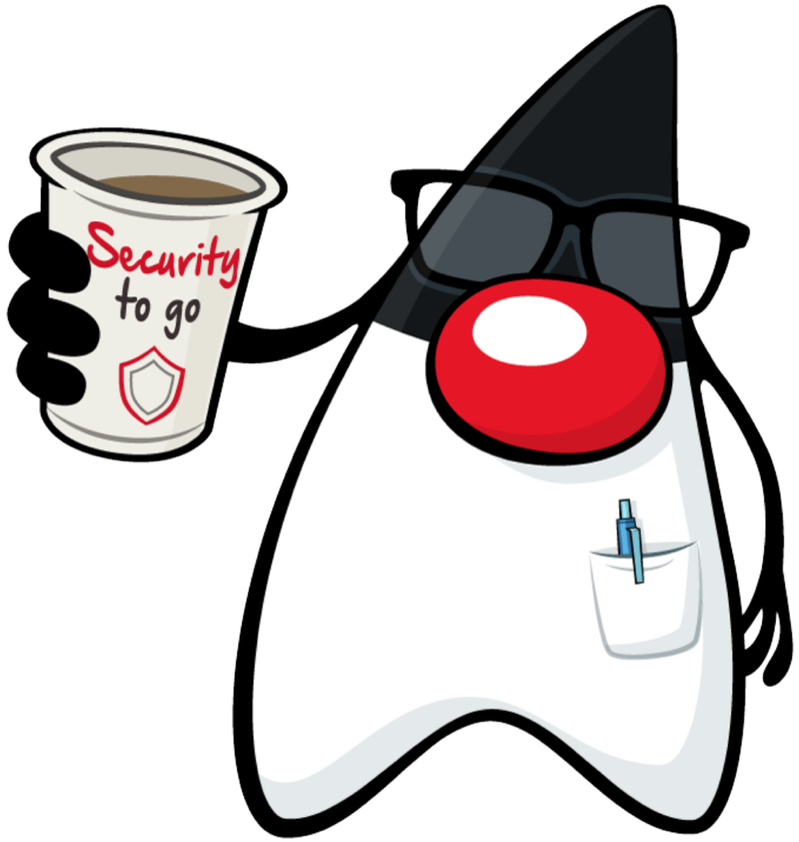
—MD5 Collision Vulnerabilities

- At Chaos Communication Congress 2008 researchers shown how they hacked MD5
- A cluster of **200 Playstation 3** has been used
- SSL certificate has been hacked
- VeriSign and others stopped using MD5



Secure Hash Algorithm (SHA)

- Family of cryptographic hash functions
- Designed by the National Security Agency (NSA)
- Published by the National Institute of Standards and Technology (NIST)
- Widely used in cryptography



Secure Hash Algorithm (SHA)

- **SHA-0:**
  - Not widely used.
  - Was quickly replaced by SHA-1 due to vulnerabilities.
- **SHA-1:**
  - Initially considered secure but later found vulnerable to collision attacks.
  - Currently, its use is deprecated in favor of more secure versions.

Secure Hash Algorithm (SHA)

- **SHA-2:**
  - Enhanced version with multiple hash sizes: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256.
  - Most popular version is SHA-256, used often in blockchain technologies.
- **SHA-3:**
  - Result of a public competition to create a new hash function.
  - Structurally different from SHA-2 but offers similar levels of security.

Secure Hash Algorithm (SHA)

- **Fixed-size Output:** Regardless of the input size, the SHA algorithm will produce a hash of fixed size specific to its version (e.g., 256 bits for SHA-256).
- **Fast Computation:** Can process data quickly, which is vital for various applications.
- **Small Changes, Big Impact:** A tiny change in input data (even one bit) should produce a significantly different hash.
- **Collision Resistant:** It should be computationally infeasible to find two different inputs that produce the same hash.

Hashing in Java

- The OpenJDK uses hashing at several places
- The OpenJDK provides different API for hash calculation

`Object.hashCode()` `HashMap` `MessageDigest`

`Hashtable` `HashSet`

Hashing in Java

- The OpenJDK uses hashing at several places
- The OpenJDK provides different API for hash calculation

`Object.hashCode()` `HashMap` **`MessageDigest`**

`Hashtable` `HashSet`

—MessageDigest Class

- A class that provides applications the functionality of a message digest algorithm for creating hash representations of data.
- The following algorithms are supported by **`MessageDigest`**

MessageDigest Class

- MD2
- MD5
- SHA-224
- SHA-256
- SHA-384
- SHA-512
- SHA-512/224
- SHA-512/256
- SHA3-224
- SHA3-256
- SHA3-384
- SHA3-512

—MessageDigest Class

```
public byte[] calculateHash(final Block block) {
    final String value = block.data() + block.previousHash();
    final byte[] message = value.getBytes(StandardCharsets.UTF_8);
    final MessageDigest md = MessageDigest.getInstance("SHA-256");
    final byte[] hash = md.digest(message);
    return hash;
}
```

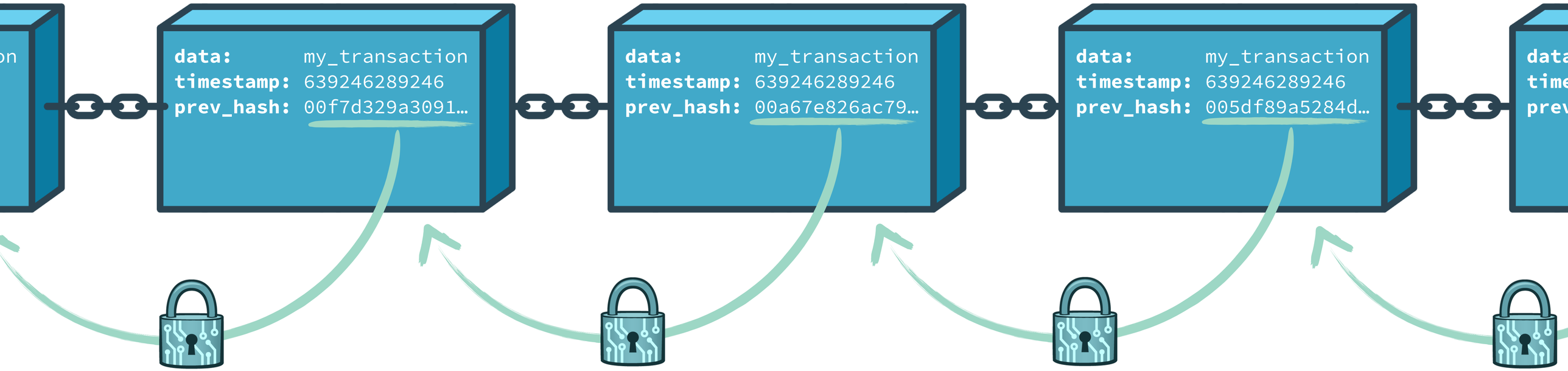
Lecture 3

Creating A Blockchain

OpenElements

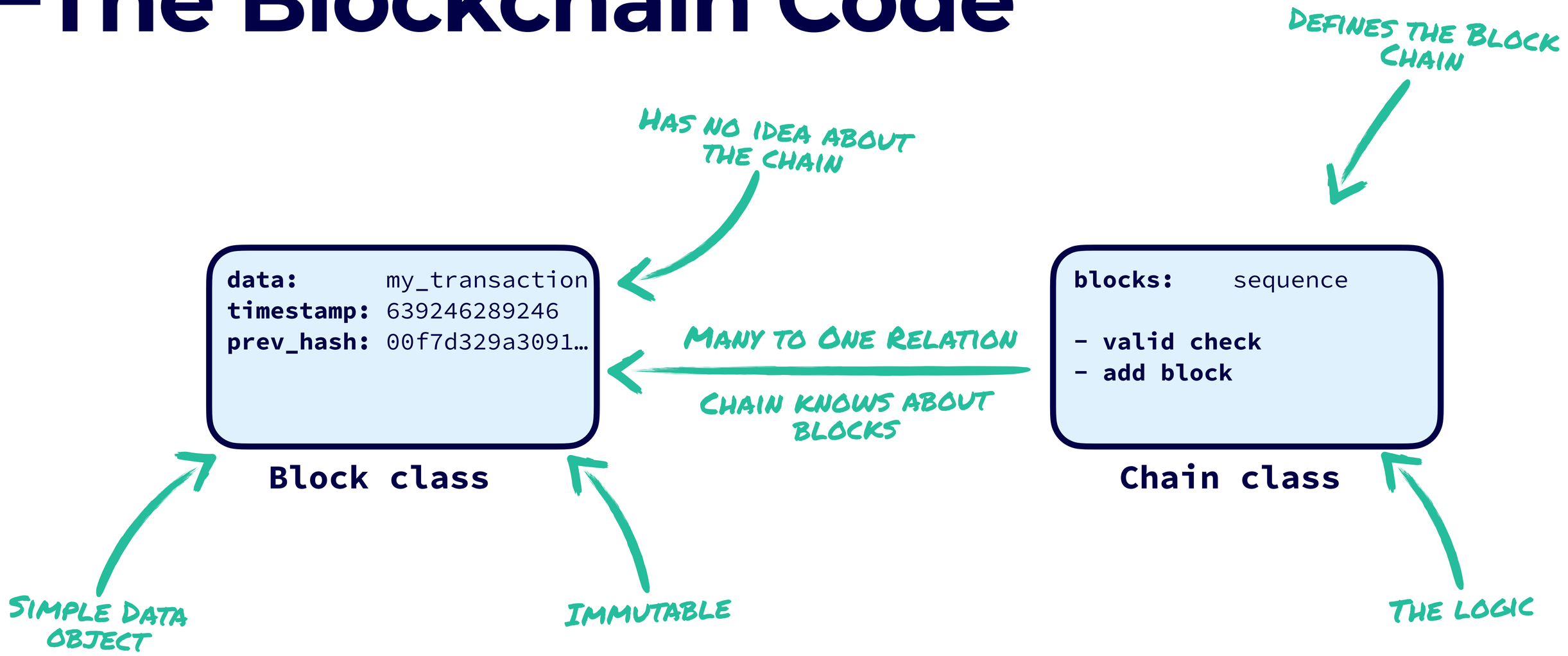
Hedera | SWIRLDS LABS

—The Blockchain



EACH BLOCKS DEFINES THE VALIDITY OF ALL PREVIOUS BLOCKS

The Blockchain Code



Let's start with a block

```
public class Block {

  private final long timestamp
  private final String data
  private final String previousHash

  public Block(final String data, final Block previousBlock) {
      this.timestamp = System.currentTimeMillis();
      this.data = data;
      this.previousHash = HashUtils.calculateHash(previousBlock);
  }

};
```

A simple block chain

```
ArrayList<Block> blockchain = new ArrayList<Block>();

Block block1 = new Block("Hi im the first block");
Block block2 = new Block("Hi im the second block“, block1);
Block block3 = new Block("Hi im the third block“, block2);

blockchain.add(block1);
blockchain.add(block2);
blockchain.add(block3);
```

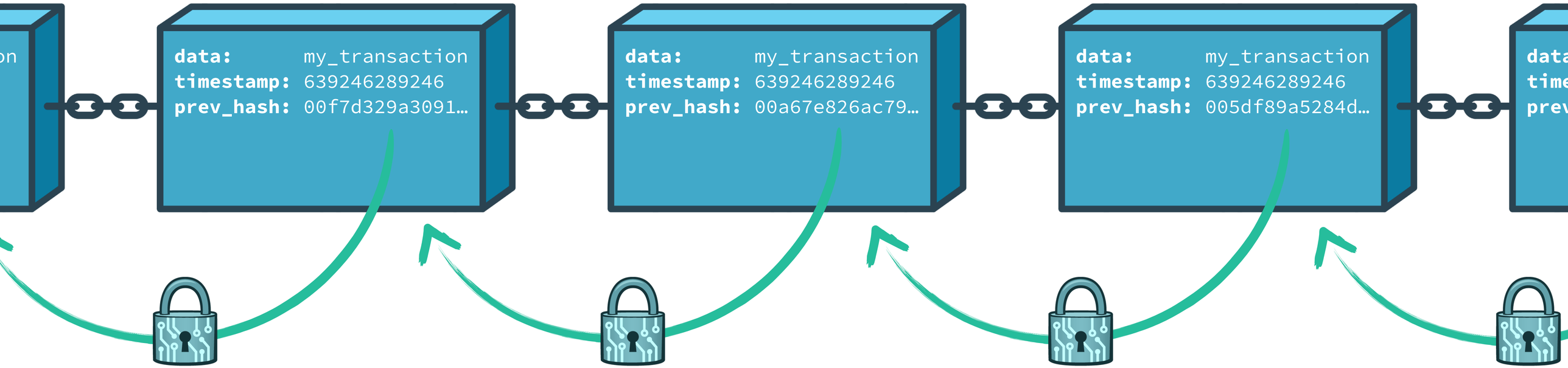
A simple block chain

```
boolean isChainValid() {
    for(int i=1; i < blockchain.size(); i++) {
        Block currentBlock = blockchain.get(i);
        Block previousBlock = blockchain.get(i-1);
        String hash = HashUtils.calculateHash(previousBlock);
        if(!Objects.equals(previousBlock.hash(), hash)) {
            return false;
        }
    }
    return true;
}
```

OpenElements

Hedera | SWIRLDS LABS

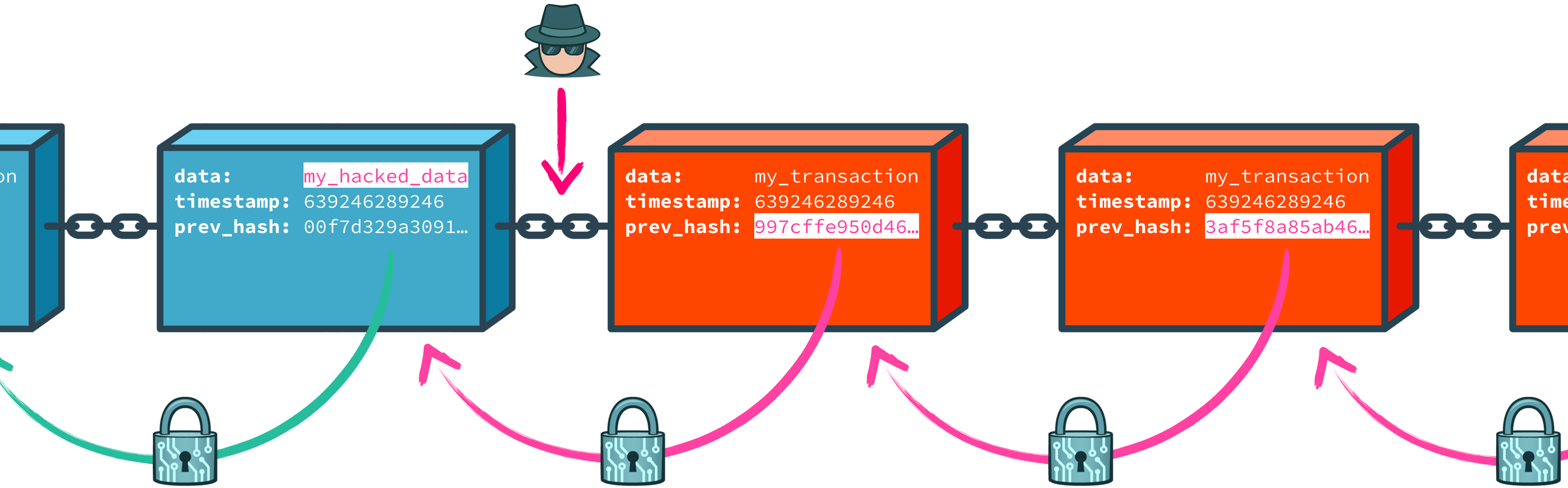
—Is that enough?



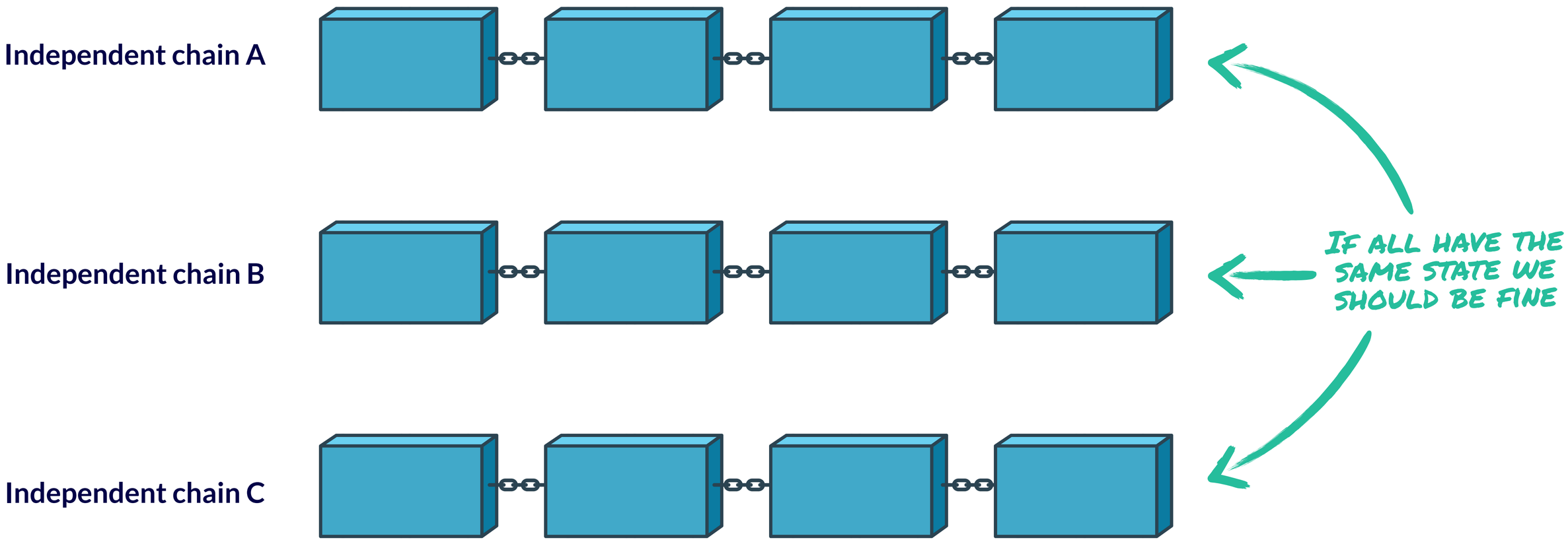
OpenElements

Hedera | SWIRLDS LABS

—Is that enough? No!

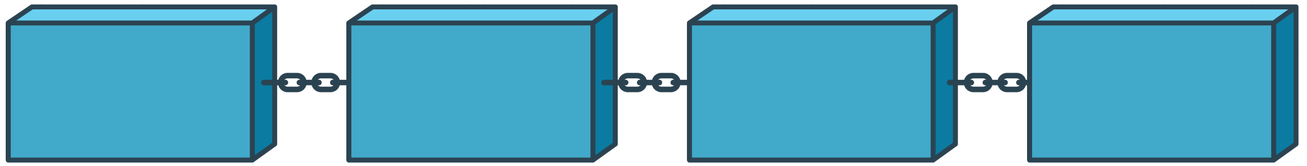


We need the same data

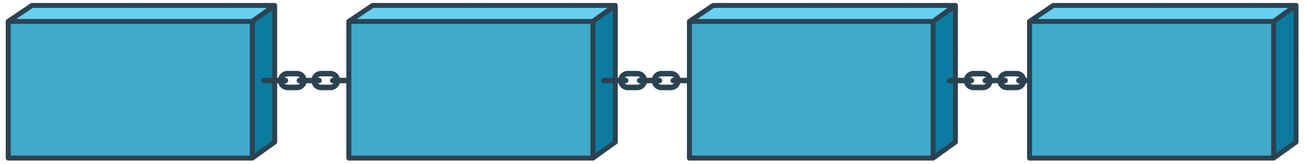


We need consensus

Independent chain A



Independent chain B



Independent chain C

